

lua-xlsx - English documentation

Julien Freyermuth

Contents

1	lua-xlsx - English documentation	1
1.1	Contents	2
1.2	Architecture and Babet integration	2
1.3	Conventions	2
1.4	Writing XLSX files	3
1.4.1	xlsx.new([opts]) -> workbook	3
1.4.2	workbook:add_sheet([name]) -> sheet	3
1.4.3	sheet:write(row, col, value [, style]) -> sheet	3
1.4.4	sheet:append_row(values [, style])	3
1.4.5	sheet:write_rows(matrix [, style])	3
1.5	Styles, layout and formulas	3
1.5.1	xlsx.VERSION	4
1.5.2	xlsx.style([opts]) -> style	4
1.5.3	sheet:set_style(row, col, style) -> sheet	5
1.5.4	sheet:set_column_width(col, width) -> sheet	5
1.5.5	sheet:set_row_height(row, height) -> sheet	5
1.5.6	sheet:freeze_panes(rows, cols) -> sheet	5
1.5.7	sheet:set_auto_filter([range]) -> sheet	6
1.5.8	xlsx.formula(expression) -> formula	6
1.5.9	Complete formatted report	6
1.5.10	workbook:save(path [, opts])	7
1.5.11	xlsx.write_rows(path, matrix [, opts])	7
1.6	1900 and 1904 dates	8
1.7	Reading XLSX files	8
1.7.1	xlsx.open(path [, opts])	8
1.7.2	workbook:date_system()	8
1.7.3	workbook:sheet_names()	8
1.7.4	workbook:sheet(which)	9
1.7.5	sheet:read(row, col)	9
1.7.6	sheet:dims()	9
1.7.7	sheet:rows()	9
1.8	ZIP validation and limits	9
1.9	DataFrame	10
1.9.1	Construction	10
1.9.2	Introspection	10
1.9.3	Non-destructive transformations	10
1.9.4	Grouping and aggregation	11
1.9.5	Output	11
1.10	Error contract	11
1.11	Tests and interoperability	11
1.12	Building the PDFs	12
1.13	Known limitations	12

1 lua-xlsx - English documentation

Reference for the `xlsx` and `dataframe` modules. Babet with Lua 5.5 is the primary target, while the pure-Lua core remains compatible with standard Lua 5.3+.

1.1 Contents

- [Architecture and Babet integration](#)
 - [Conventions](#)
 - [Writing XLSX files](#)
 - [Styles, layout and formulas](#)
 - [1900 and 1904 dates](#)
 - [Reading XLSX files](#)
 - [ZIP validation and limits](#)
 - [DataFrame](#)
 - [Error contract](#)
 - [Tests and interoperability](#)
 - [Building the PDFs](#)
 - [Known limitations](#)
-

1.2 Architecture and Babet integration

`xlsx.lua` and `dataframe.lua` require no external Lua module. When the global `babet` table is available, `xlsx.lua` automatically uses:

Babet function	lua-xlsx use
<code>babet.writeFileAtomic</code>	atomic and durable XLSX publication
<code>babet.archive.test</code>	full ZIP validation before parsing
<code>babet.fileSize</code>	file-size check before loading into memory
<code>babet.crc32</code>	native CRC-32 calculation

If one function is absent, the pure-Lua path is retained for that operation.

To disable Babet for one call:

```
assert(wb:save("report.xlsx", { use_babet = false })))
```

```
local read, err = xlsx.open("report.xlsx", {  
    use_babet = false,  
})  
assert(read, err)
```

This is mostly useful for testing the fallback. Production code running in Babet should normally keep the default `true`.

1.3 Conventions

- `sheet:write(row, col, value [, style])` and `sheet:read(row, col)` use **0-based** indices. Cell A1 is (0, 0).
 - `sheet:rows()` yields **1-based** Lua arrays. `row[1]` is column A and `row.n` is the width.
 - An empty cell is `nil`. Boolean `false` never collapses to `nil`.
 - Dates are returned as ISO 8601 strings.
 - XLSX bounds are enforced: rows 0..1048575, columns 0..16383.
 - Written strings must be valid UTF-8 and contain only XML 1.0 characters.
 - `DataFrame` transformations return newly copied row records.
-

1.4 Writing XLSX files

1.4.1 `xlsx.new([opts]) -> workbook`

Option:

Option	Default	Values
<code>date_system</code>	"1900"	"1900" or "1904"

```
local wb1900 = xlsx.new()
local wb1904 = xlsx.new({ date_system = "1904" })
```

Unknown options are rejected.

1.4.2 `workbook:add_sheet([name]) -> sheet`

The default name is SheetN.

Rules:

- 1 to 31 **Unicode characters**, not bytes;
- valid UTF-8 and XML 1.0;
- none of `:`, `\`, `/`, `?`, `*`, `[` or `]`;
- no leading or trailing apostrophe;
- no exact duplicate or ASCII case-only duplicate.

```
local wb = xlsx.new()
local data = wb:add_sheet("Data")
local unicode = wb:add_sheet(string.rep("é", 20))
```

1.4.3 `sheet:write(row, col, value [, style]) -> sheet`

Accepted values: string, finite number, boolean, date value, formula value or `nil`. The optional style must be created by `xlsx.style`.

```
local sh = xlsx.new():add_sheet("Example")
sh:write(0, 0, "Text")
sh:write(0, 1, 42)
sh:write(0, 2, 3.14)
sh:write(0, 3, false)
sh:write(0, 4, xlsx.date(2026, 7, 18))
sh:write(0, 5, xlsx.formula("SUM(B1:C1)"))
```

NaN, infinities, unsupported types and out-of-range indices raise a Lua error.

1.4.4 `sheet:append_row(values [, style])`

```
sh:append_row({ "Alice", 30, true })
sh:append_row({ "Bob", 25, false })
```

1.4.5 `sheet:write_rows(matrix [, style])`

```
sh:write_rows({
  { "Name", "Score" },
  { "Alice", 18 },
  { "Bob", 15 },
})
```

1.5 Styles, layout and formulas

The APIs in this section affect writing. The reader returns cell values but does not yet expose styles, dimensions, filters, frozen panes or formula expressions.

1.5.1 `xlsx.VERSION`

```
assert(xlsx.VERSION == "1.1.0")
```

1.5.2 `xlsx.style([opts]) -> style`

Creates an immutable reusable style. `sheet:write()` rejects raw option tables; styles must be created through this constructor.

Option	Type	Values
<code>bold</code>	boolean	bold font
<code>italic</code>	boolean	italic font
<code>font_color</code>	string	RGB RRGGBB, #RRGGBB or ARGB AARRGGBB
<code>fill_color</code>	string	solid RGB or ARGB fill
<code>horizontal</code>	string	left, center, right, justify
<code>vertical</code>	string	top, center, bottom, justify
<code>wrap_text</code>	boolean	automatic text wrapping
<code>number_format</code>	string	documented alias or Excel format code

Six-digit RGB colors automatically receive an opaque FF alpha channel.

Bold font:

```
local bold = xlsx.style({ bold = true })
sh:write(0, 0, "Important", bold)
```

Italic colored font:

```
local note = xlsx.style({
  italic = true,
  font_color = "7F6000",
})
sh:write(1, 0, "Note", note)
```

Solid fill:

```
local warning = xlsx.style({ fill_color = "FFF2CC" })
sh:write(2, 0, "Warning", warning)
```

Horizontal alignment:

```
local centered = xlsx.style({ horizontal = "center" })
sh:write(3, 0, "Centered", centered)
```

Vertical alignment and wrapping:

```
local wrapped = xlsx.style({
  vertical = "center",
  wrap_text = true,
})
sh:write(4, 0, "Long text split over several lines", wrapped)
```

Built-in number-format aliases:

Alias	Generated Excel code
<code>general</code>	no custom format
<code>integer</code>	0
<code>decimal</code>	0.00
<code>percent</code>	0.00%
<code>currency_eur</code>	#,##0.00 "€"
<code>currency_usd</code>	\$#,##0.00
<code>date</code>	yyyy-mm-dd
<code>datetime</code>	yyyy-mm-dd hh:mm:ss

One example per common format:

```
sh:write(0, 1, 42, xlsx.style({ number_format = "integer" }))
sh:write(1, 1, 3.14159, xlsx.style({ number_format = "decimal" }))
sh:write(2, 1, 0.125, xlsx.style({ number_format = "percent" }))
sh:write(3, 1, 19.90, xlsx.style({ number_format = "currency_eur" }))
```

Custom Excel format codes are accepted:

```
local weight = xlsx.style({ number_format = '0.000 "kg"' })
sh:write(4, 1, 12.3456, weight)
```

xlsx.date and xlsx.datetime automatically add their date format when the provided style has no number_format. This allows font and fill styling without losing date semantics:

```
local date_bold = xlsx.style({ bold = true })
sh:write(0, 2, xlsx.date(2026, 7, 18), date_bold)
```

Combined header style:

```
local header = xlsx.style({
  bold = true,
  font_color = "FFFFFF",
  fill_color = "4472C4",
  horizontal = "center",
  vertical = "center",
  wrap_text = true,
})
sh:append_row({ "Product", "Price", "Date" }, header)
```

1.5.3 sheet:set_style(row, col, style) -> sheet

Applies a style after writing, or to an empty cell:

```
sh:write(0, 0, "Title")
sh:set_style(0, 0, xlsx.style({ bold = true }))
sh:set_style(1, 0, xlsx.style({ fill_color = "E2F0D9" }))
```

Passing nil removes the recorded style:

```
sh:set_style(0, 0, nil)
```

1.5.4 sheet:set_column_width(col, width) -> sheet

col is 0-based. Width must be finite and between 0.1 and 255 Excel units.

```
sh:set_column_width(0, 24)
sh:set_column_width(1, 14)
```

1.5.5 sheet:set_row_height(row, height) -> sheet

row is 0-based. Height is measured in points and must be between 0.1 and 409.5.

```
sh:set_row_height(0, 28)
```

1.5.6 sheet:freeze_panes(rows, cols) -> sheet

rows and cols are the counts kept visible from the top-left corner.

Freeze the first row:

```
sh:freeze_panes(1, 0) -- first free cell: A2
```

Freeze the first column:

```
sh:freeze_panes(0, 1) -- first free cell: B1
```

Freeze both:

```
sh:freeze_panes(1, 1) -- first free cell: B2
```

freeze_panes(0, 0) clears the setting.

1.5.7 sheet:set_auto_filter([range]) -> sheet

Without an argument, the used range is selected when the workbook is saved:

```
sh:set_auto_filter()
```

An explicit rectangular A1 range is also accepted:

```
sh:set_auto_filter("A1:D100")
```

Passing false disables the filter:

```
sh:set_auto_filter(false)
```

This adds Excel filter buttons; it does not remove or hide rows itself.

1.5.8 xlsx.formula(expression) -> formula

Creates a formula for writing. A leading = is optional:

```
sh:write(3, 1, xlsx.formula("SUM(B1:B3)"))
sh:write(3, 2, xlsx.formula("=AVERAGE(C1:C3)"))
```

lua-xlsx stores the formula but does not calculate it and does not write a cached result. Excel, LibreOffice or another spreadsheet application calculates it when opening the workbook. A data_only reader may therefore see nil until a spreadsheet program recalculates and saves the file.

A formula accepts the same optional style as any other value:

```
local money = xlsx.style({ number_format = "currency_eur" })
sh:write(10, 3, xlsx.formula("SUM(D2:D10)", money))
```

1.5.9 Complete formatted report

```
local xlsx = require("xlsx")
```

```
local wb = xlsx.new()
```

```
local sh = wb:add_sheet("Sales")
```

```
local header = xlsx.style({
    bold = true,
    font_color = "FFFFFF",
    fill_color = "4472C4",
    horizontal = "center",
    vertical = "center",
    wrap_text = true,
})
```

```
local money = xlsx.style({ number_format = "currency_eur" })
```

```
sh:append_row({ "Product", "Price", "Quantity", "Total" }, header)
sh:append_row({ "Keyboard", 39.90, 2 })
sh:append_row({ "Mouse", 24.50, 3 })
```

```
sh:write(1, 1, 39.90, money)
sh:write(2, 1, 24.50, money)
sh:write(1, 3, xlsx.formula("B2*C2"), money)
sh:write(2, 3, xlsx.formula("B3*C3"), money)
sh:write(3, 3, xlsx.formula("SUM(D2:D3)"), money)
```

```
sh:set_column_width(0, 24)
sh:set_column_width(1, 14)
```

```

sh:set_column_width(2, 12)
sh:set_column_width(3, 16)
sh:set_row_height(0, 28)
sh:freeze_panes(1, 1)
sh:set_auto_filter("A1:D3")

assert(wb:save("sales.xlsx"))

```

1.5.10 workbook:save(path [, opts])

Option	Default	Effect
overwrite	true	replace an existing file
durable	true	request durable synchronization in Babet
permissions	0644	final Babet file mode
use_babet	true	use writeFileAtomic when available

Simple write:

```

local ok, err = wb:save("report.xlsx")
assert(ok, err)

```

Refuse overwrite:

```

local ok, err = wb:save("report.xlsx", {
    overwrite = false,
})

```

Rebuildable cache:

```

assert(wb:save("cache.xlsx", {
    durable = false,
}))

```

Private permissions:

```

assert(wb:save("private.xlsx", {
    permissions = tonumber("600", 8),
}))

```

Combined example:

```

local ok, err = wb:save("export/report.xlsx", {
    overwrite = true,
    durable = true,
    permissions = tonumber("640", 8),
    use_babet = true,
})
assert(ok, err)

```

With Babet, this calls `babet.writeFileAtomic`. Standard Lua writes a temporary file in the same directory, checks write, flush and close, then renames it. The fallback is atomic on ordinary POSIX filesystems but does not duplicate all of Babet's descriptor confinement and `fsync` guarantees.

1.5.11 xlsx.write_rows(path, matrix [, opts])

Options are sheet, date_system, overwrite, durable, permissions and use_babet.

```

assert(xlsx.write_rows("result.xlsx", {
    { "x", "y" },
    { 1, 2 },
    { 3, 4 },
}, {

```

```

    sheet = "Results",
    date_system = "1900",
    overwrite = true,
  )))

```

1.6 1900 and 1904 dates

```

sh:write(0, 0, xlsx.date(2026, 7, 18))
sh:write(0, 1, xlsx.datetime(2026, 7, 18, 13, 45, 30))

```

The constructors validate real calendar dates, leap years, years 1..9999, hours 0..23, minutes and seconds 0..59, and integer argument types.

Helpers:

```

local serial1900 = xlsx.date_to_serial(2026, 7, 18)
local serial1904 = xlsx.date_to_serial(2026, 7, 18, 0, 0, 0, true)

print(xlsx.serial_to_iso(serial1900, false))
print(xlsx.serial_to_iso(serial1904, false, true))

```

The writer emits the correct serial for the workbook's date_system. The reader automatically detects <workbookPr date1904="1"/>.

1.7 Reading XLSX files

1.7.1 xlsx.open(path [, opts])

Opening performs:

1. file-size inspection;
2. babet.archive.test() validation when available;
3. bounded in-memory loading;
4. central and local ZIP header validation;
5. CRC-checked extraction of the XML parts that are used;
6. workbook, shared-string and style parsing.

```

local wb, err = xlsx.open("report.xlsx")
assert(wb, err)

```

Disable only Babet's archive preflight:

```

local wb, err = xlsx.open("report.xlsx", {
  validate_archive = false,
})

```

The Lua ZIP checks still apply.

1.7.2 workbook:date_system()

Returns "1900" or "1904".

1.7.3 workbook:sheet_names()

```

for _, name in ipairs(wb:sheet_names()) do
  print(name)
end

```


1.7.4 workbook:sheet(which)

which is a name or a 1-based integer index.

```
local first = wb:sheet(1)
local data = wb:sheet("Data")
```

Worksheets are extracted lazily and cached. A worksheet-specific corruption is therefore returned by sheet() as nil, err.

1.7.5 sheet:read(row, col)

Returns a string, number, boolean, ISO date string or nil. OOXML error cells are currently returned as strings.

1.7.6 sheet:dims()

Returns maximum 0-based row and column indices, or -1, -1 for an empty sheet.

1.7.7 sheet:rows()

```
for row in sheet:rows() do
  for col = 1, row.n do
    print(row[col])
  end
end
```

Intermediate empty spreadsheet rows are yielded to preserve row numbering.

1.8 ZIP validation and limits

Option	Default	Maximum
max_file_size	256 MiB	no extra internal ceiling
max_entries	10,000	100,000
max_entry_size	64 MiB	8 GiB
max_total_size	512 MiB	64 GiB
max_path_length	4,096 bytes	1 MiB
max_total_name_bytes	16 MiB	64 MiB
max_compression_ratio	200	1,000,000,000
use_babet	true	boolean
validate_archive	true	boolean

One example per limit:

```
xlsx.open("a.xlsx", { max_file_size = 32 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_entries = 2000 })
xlsx.open("a.xlsx", { max_entry_size = 16 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_total_size = 128 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_path_length = 1024 })
xlsx.open("a.xlsx", { max_total_name_bytes = 1024 * 1024 })
xlsx.open("a.xlsx", { max_compression_ratio = 100 })
```

Combined untrusted-file profile:

```
local wb, err = xlsx.open("import.xlsx", {
  max_file_size = 64 * 1024 * 1024,
  max_entries = 5000,
  max_entry_size = 16 * 1024 * 1024,
  max_total_size = 128 * 1024 * 1024,
  max_path_length = 2048,
```

```

max_total_name_bytes = 2 * 1024 * 1024,
max_compression_ratio = 100,
use_babet = true,
validate_archive = true,
})
assert(wb, err)

```

The Lua reader rejects multi-disk ZIPs, ZIP64, encryption, unsupported methods, unsafe or duplicate names, inconsistent local headers or data descriptors, overlapping entries, size mismatches, CRC failures and truncated DEFLATE data.

1.9 DataFrame

A DataFrame contains unique, non-empty string columns and rows records.

1.9.1 Construction

1.9.1.1 DF.from_rows(matrix [, opts]) opts.header uses the first row as names. opts.columns supplies explicit names when header is false.

```

local d = DF.from_rows({
  { "name", "age" },
  { "Alice", 30 },
  { "Bob", 25 },
}, { header = true })

```

Duplicate column names are rejected.

1.9.1.2 DF.from_records(records [, columns])

```

local d = DF.from_records({
  { name = "Alice", age = 30 },
  { name = "Bob", age = 25 },
}, { "name", "age" })

```

Without columns, string keys are inferred and sorted.

1.9.1.3 DF.from_sheet(sheet [, opts])

```

local d = DF.from_sheet(assert(wb:sheet("Data")), {
  header = true,
})

```

1.9.2 Introspection

```

print(d:nrow(), d:ncol())
print(table.concat(d:colnames(), ", "))
local ages = d:column("age")

```

d:iter() yields copies:

```

for row in d:iter() do
  row.age = 0 -- does not modify d
end

```

1.9.3 Non-destructive transformations

```

local adults = d:filter(function(row) return row.age >= 18 end)
local names = d:select("name")
local renamed = d:rename({ age = "years" })
local doubled = d:mutate("double_age", function(row) return row.age * 2 end)
local asc = d:sort("age")

```

```

local desc = d:sort("age", { desc = true })
local first = d:head(10)
local last = d:tail(10)

```

filter and mutate callbacks receive a copy. filter, sort, head, tail and iter do not share row tables with the source. Rename collisions and repeated selections are rejected. Sort is stable and keeps nil last.

1.9.4 Grouping and aggregation

```

local grouped = sales:groupby("city", "product"):agg({
  total = { "sum", "amount" },
  average = { "mean", "amount" },
  minimum = { "min", "amount" },
  maximum = { "max", "amount" },
  first = { "first", "amount" },
  last = { "last", "amount" },
  count = { "count" },
})

```

Functions: sum, mean/avg, min, max, count, first, last.

Group keys support nil, booleans, binary strings and numbers. The internal key encoding is type- and length-prefixed, so embedded separators or NUL bytes cannot merge distinct groups.

```

local counts = d:groupby("city"):count()

```

1.9.5 Output

```

local records = d:to_records()
local rows = d:to_rows()
local no_header = d:to_rows({ header = false })
print(d:tostring(20))
d:show(20)

```

1.10 Error contract

Caller mistakes raise Lua errors: invalid types or options, invalid names or dates, out-of-range indices and inconsistent DataFrame operations.

Filesystem and data failures normally return nil, err:

```

local wb, err = xlsx.open("missing.xlsx")
if not wb then
  io.stderr:write(err, "\n")
end

```

workbook:sheet() follows the same convention for a corrupt or missing sheet part. Workbook:save() checks writing, flushing, closing and publication and no longer reports success after a real write failure.

1.11 Tests and interoperability

```
./run_tests.sh
```

The harness covers write/read round trips, Unicode, XML entities, CRC corruption, 1900/1904 dates, styles and number formats, dimensions, frozen panes, filters, formulas, Excel and XML limits, Babet calls, DataFrame copy semantics, collision-free grouping and real openpyxl interoperability.

Babet is resolved in this order: BABET_BIN, the local bin/babet file, then babet from PATH. The harness creates a temporary Python virtual environment, installs openpyxl<=3.1,>4, runs the

interoperability round trip separately with Babet and standard Lua when both are available, and removes the venv, pip cache and temporary files at exit, including after a failure. A global openpyxl installation is neither used nor required.

This phase requires python3, the venv module, pip inside the venv and access to the configured Python package index. Debian and Ubuntu systems may need the python3-venv package.

```
cp /path/to/babet bin/babet
chmod +x bin/babet
./run_tests.sh
```

```
BABET_BIN=../babet/bin/babet ./run_tests.sh
LUA_BIN=/usr/bin/lua5.5 ./run_tests.sh
OPENPYXL_SPEC='openpyxl==3.1.5' ./run_tests.sh
```

1.12 Building the PDFs

```
cd doc
./build_doc.sh
```

Outputs:

- documentation-fr.pdf;
- documentation-en.pdf.

Pandoc and XeLaTeX or LuaLaTeX are required. Auxiliary LaTeX files are kept in a temporary directory.

1.13 Known limitations

- Writing uses STORED ZIP entries; reading supports STORED and DEFLATE.
- The internal Lua parser does not support ZIP64.
- No merged cells, charts, images, data validation or conditional formatting.
- Formulas can be written, but lua-xlsx does not calculate them and does not yet expose their expression while reading. Only an optional cached value is read.
- Styles, dimensions, panes and filters are written but are not exposed by the reading API.
- No streaming reader: the file and selected XML parts remain in memory.
- The XML scanner is XLSX-specific, not a general XML parser.
- No Unicode normalization or complete Unicode case folding for sheet names; case-insensitive duplicate detection is reliable for ASCII.